Homework 4
Due at 11:59 p.m. on Fri. Oct. 23

By handing in the homework you are agreeing to the Homework Rules; see Ed Discussion.

1. Let A be an integer array of length n. Each element of A is distinct. Assume n is divisible by 2. Exactly half the integers in A are even numbers, and at least half are odd numbers. Consider the following counting procedure:

```
count = 0
for i=1 to n-1
   if A[i] == A[i+1] mod 2  // if they are both even or both odd
        count++
return count
```

Suppose the n elements of array A are randomly permuted, so that each possible permutation is equally likely. If we run the above counting procedure on the resulting random array A, what is the expected value of count when it is returned?

Express your answer as a function of n. Give an exact answer, eon't use asymptotic notation. [Hint: Use indicator variables and linearity of expectation.]

2. The textbook presents the pseudocode for Partition(A,p,r). It partitions the elements in A[p:r] around a pivot element x, which is taken to be A[r], the last element in the subarray.

The pseudocode for Partition is in the textbook, but let's just focus here on what it does, and not on exactly how it works (although you can check that out if you have time).

Partition(A,p,r) rearranges the elements in A[p:r] so that the elements that are $\leq$ x all appear to the left of x (in lecture, we called these the elements in Low), and the elements $>$ x all appear to the right of x (we called these the elements in High). It then returns the index containing x.

Below is the pseudocode for Quicksort.

```
QUICKSORT(A, p, r)

if p < r
  // Partition the subarray around the pivot A[r], which ends up in A[q].
   q = PARTITION(A, p, r)

   QUICKSORT(A, p, q - 1)

   QUICKSORT(A, q + 1, r)
```

What is the worst-case running time of the textbook's version of Quicksort if all elements in A are equal? Express your answer in theta notation, as a function of n, the length of A.

3. Consider the problem of merging two sorted lists,one of length n, and the other of length 2.

(a) Consider the following standard merge procedure, which merges sorted array A of length 2 with sorted array B of length n, into a third array C of length $n + 2$.

```
Standard-Merge(A,B,C,n)

i = 1
j = 1
k = 1
while i <= n and j <= 2
     if A[i] <= B[j]
          C[k]=A[i]
          i = i+1
     else
         C[k]=B[j]
         j = j+1
     k = k+1
// Having exhausted the elements in either A or B, put remaining elements
// of other array into C
while j <= 2
    C[k]=B[j]
    j=j+1
    k=k+1
while i <= n
    C[k]=A[i]
    i=i+1
    k=k+1
```

Show the decision tree that illustrates the comparisons that would be performed by running Standard-Merge on input arrays A=$[a_1, a_2]$ and B=$[b_1,b_2]$, and the ordering of the elements in the final merged array (so $n = 2$ and C has length 4). The non-leaf nodes of your tree should each contain a comparison (e.g., the root of your tree should have the comparison $a_1 < b_1$). The leaves of your tree should each contain an ordering of the elements a_1, a_2, b_1, b_2.

(b) Assume all elements of A and B are distinct. How many comparisons does the above algorithm make in the worst case? Count only comparisons between array elements. Give the *exact* expression as a function of n. Do not use asymptotic notation in your answer.

(c) Use logic similar to that used in obtaining the $\Omega(n \log n)$ decision tree lower bound for sorting, to argue that *any* comparison-based algorithm needs to make at least X comparisons in the worst case to perform the merge. You will determine X by answering the following questions.

i. Imagine a decision tree corresponding to a comparison-based algorithm that solves this problem. How many leaves does it need to have?

ii. What is the minimum height of such a decision tree, given your answer to the previous question? Call this quantity X. If you answered this question correctly, then you just showed that any comparison based algorithm for this problem needs to make at least X queries in the worst case.

(d) Design an algorithm for this problem whose worst-case number of comparisons is $o(n)$ (that's little-oh of n). Count only the number of comparisons between array elements.

(e) Is your answer to the previous question $O(X)$ (big-Oh of X)? In other words, does the worst-case number of comparisons you proved for your algorithm match (asymptotically) the lower bound X you proved above?